Confluent Kafka Isotope

An example of using Kafka **consumer and producer interceptors** to tag every message with a tracer — an *isotope* — that carries through every hop of a multi-topic pipeline. Apache Flink will eventually consume the tagged records and report on what the isotopes reveal: end-to-end latency, hop topology, drop/duplication rates, and pipeline coverage.

A **portability requirement** runs through this project: the same isotope mechanism must work against both **Confluent Cloud for Apache Flink** (managed; restricted to Table API SQL + uploaded UDF/PTF JARs) and **Confluent Platform for Apache Flink** (full DataStream + Table API). Three decisions follow from that: tagging happens in the Kafka **producer/consumer interceptors** (the one extension point both runtimes share via the broker), the on-wire format is **JSON in record headers** (so Flink SQL can read the scalar fields with `CAST(headers[…] AS STRING)` and no UDF), and the optional stateful reports (`LatencyPercentilesUDAF`, `StuckTracePTF`) ship as a single JAR that registers identically on either runtime.

How the isotope is carried

- **Header `x-isotope`** (JSON bytes) carries the full hop history, forwarded by every hop:
  - `t` — 16-byte random trace ID, stable for the life of the trace
  - `o` — origin timestamp (ms)
  - `s` — origin service name (set once, never reassigned)
  - `h` — ordered list of hops, each `{s: service, t: topic, m: tsMs}`
  - `x` — `true` if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted
- **Six scalar headers** (UTF-8 strings) carry the most-recent-hop view so Flink SQL can read them via `CAST(headers['x-isotope-…'] AS STRING)` without parsing the JSON array (no UDF required on either CCAF or CP Flink). See flink/README.md for the full header table.

A producer with the isotope interceptor loaded appends one hop on every `send()`. A consume-then-produce service calls `IsotopeContext.adoptFromRecord(record)` between consume and produce so the trace ID and origin survive the hop.

Repo layout

```
app/                                    isotope JVM library + tests
  src/main/java/com/life360/kafka/isotope/
    Isotope.java                        POJO + JSON codec + Hop +
fromHeaders()
    IsotopeContext.java                 ThreadLocal + adoptFromRecord()
    IsotopeProducerInterceptor.java     stamps/appends x-isotope + 6 scalar
                                        reporting headers on send()
    IsotopeConsumerInterceptor.java     batch-aware logging; no auto-
propagation
  src/test/java/.../                    IsotopeCodecTest (no broker needed)
  src/integrationTest/java/.../         live-broker tests (need Minikube
CP)
ptf/                                    Phase 2 — Flink PTF + UDAF shadow
```

```
JAR
  src/main/java/com/life360/kafka/isotope/flink/
    LatencyPercentilesUDAF.java         T-Digest p50/p95/p99 aggregate
    StuckTracePTF.java                  per-trace state + event-time timer
    Percentiles.java, StuckTraceAlert.java  return-type DTOs
  src/test/java/.../                    LatencyPercentilesUDAFTest
flink/sql/                              Flink SQL reports — identical on
                                        CCAF and CP Flink except source DDL
  cp/, cc/, shared/                     per-environment source + shared
reports
k8s/base/                               CFK Kafka/SR/Connect/ksqlDB/C3
manifests
scripts/                                port-forward helpers,
dump_cc_topic.py
Makefile                                cp-up / flink-up / kafka-pf-up /
...
```

Running

Unit tests (no broker needed)

```
./gradlew :app:test
```

Integration tests (live Kafka via Minikube)

```
make minikube-start         # one-time
make cp-up                  # CFK Operator + Kafka/SR/Connect/ksqlDB/C3
make kafka-pf-up            # port-forward CFK NodePort listeners to
localhost:30092
./gradlew :app:integrationTest
make kafka-pf-down          # when done
```

The integration tests cover:

Test	What it verifies
`BrokerSmokeIT`	AdminClient can create/list/delete a topic via the NodePort path
`ProducerInterceptorIT`	A bare consumer sees the `x-isotope` header with the expected origin/hop
`ConsumerInterceptorIT`	`adoptFromRecord` extracts isotope into thread-local; clears for untagged records
`ThreeStageHopPropagationIT`	`svc-A → topic-AB → svc-B → topic-BC → svc-C` produces a 2-hop trace with stable trace ID

Status

Piece	Status
Kafka interceptors + JSON codec	Implemented; unit tests passing
Live-broker integration tests	Written; awaiting first cluster run
Flink SQL reporting (CC + CP)	Phase 1 — four reports written; awaiting first deploy
Demo pipeline (Stage1/2/3 services)	Not yet implemented; the ITs play the role of the demo for now
PTF stuck-trace + UDAF percentiles	Phase 2 — JAR builds, UDAF unit-tested, two SQL reports written; PTF awaits first live-cluster run for end-to-end verification